DSA0309-NATURAL LANGUAGE PROCESSING

ASSIGNMENT-2

Name:T.Gayathri

Regno:192425239

ASSIGNMENT 2 – CO2

CO2 Coverage: N-grams Models of Syntax; Counting Words; Unsmoothed N-grams; Smoothing — Backoff, Deleted Interpolation; Entropy; English Word Classes; Tagsets for English; Part of Speech Tagging; Rule-Based Part of Speech Tagging; Stochastic Part of Speech Tagging; Transformation-Based Tagging

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Submission Date: 8-11-2026

Assignment Title: Build and Evaluate an N-gram Language Model and a Part-of-Speech Tagger for a Public Transport Enquiry Helpline (Total Marks – 50)

Duration: 1 week

TASK 1 – Counting Words and Building the Vocabulary (3 Marks)

1. Introduction

Word counting is an important step in Natural Language Processing (NLP). It helps identify the words used in a transport enquiry helpline. The collected words are used to build a vocabulary and train language models.

2. Types of Word Counting

Term	Meaning	Example
Type	Unique word in a corpus	bus
Token	Every occurrence of a word	bus, bus
Lemma	Basic dictionary form	booking → book

Word form	Actual form used in a sentence	book, books, booking

3. Tokenisation Decisions

Tokenisation means dividing sentences into individual words.

Example sentence:

"Book two tickets to Vellore at 6:30."

Tokenisation decisions:

Convert uppercase letters into lowercase.

Remove unnecessary punctuation.

Treat numbers as individual tokens.

Treat 6:30 as one token.

Convert different forms into their basic form when calculating lemmas.

Use <s> and </s> to identify sentence boundaries.

Example:

Book two tickets to Vellore

Tokens:

book, two, tickets, to, vellore

4. Difference Between Tokenisation Methods

Method	**Example**	**Vocabulary effect**
Case-sensitive	Bus, bus	Two types
Lowercase	Bus, bus	One type
Time separated	6:30 → 6, 30	More tokens
Time combined	6:30	One token

Lowercase conversion and consistent treatment of numbers reduce unnecessary vocabulary differences.

5. Zipf's Law

Zipf's Law states that a frequently occurring word has a higher rank, while less frequently occurring words have a lower frequency.

The formula is:

\ f(r)≈ (C)/(r) \

Where:

\f(r)\ = Frequency of a word.

\r\ = Rank of the word.

\C\ = Constant.

Example:

Word	Rank	Frequency
the	1	5000
bus	2	2500
to	3	1600
ticket	4	1200
train	5	1000

Observation: Common words occur frequently, while station names and special words occur less frequently.

6. Vocabulary and Unknown Words

Vocabulary is the collection of unique words present in the training corpus.

Rare words can be replaced with <UNK> (unknown word).

Example:

Vellore, Chennai, Madurai, Hyderabad

If a word occurs only once and the threshold is 2, it can be replaced with <UNK>.

Threshold	Meaning
1	Keep all observed words
2	Replace words occurring fewer than 2 times
3	Replace words occurring fewer than 3 times
5	Replace words occurring fewer than 5 times

The actual vocabulary sizes and percentages must be calculated from the training corpus.

7. Out-of-Vocabulary (OOV) Rate

OOV means words that occur in the development dataset but are not present in the training vocabulary.

Formula:

\ OOV\ Rate=\frac{\text{Unknown words}}{\text{Total words}}×100 \

Example:

If 100 words are tested and 10 words are unknown:

\ OOV=(10)/(100)×100=10\% \

Result: Word counting and vocabulary building help the language model identify common and unknown words. Correct tokenisation improves model performance.

TASK 2 – Unsmoothed N-gram Models (4 Marks)

1. Introduction

An N-gram model is a language model that predicts the next word based on the previous words.

It is used to identify the most suitable sentence from multiple speech recognition outputs.

2. Chain Rule of Probability

The chain rule calculates the probability of a complete sentence using the probabilities of individual words.

\ P(w_1,w_2,\ldots,w_n) \

\ =P(w_1)P(w_2|w_1)\cdots P(w_n|w_1,\ldots,w_{n-1}) \

3. Types of N-gram Models

Unigram Model

It considers only one word at a time.

Example: "book", "bus", "ticket".

\ P(w)=(Count(w))/(Total\ words) \

Bigram Model

It considers two consecutive words.

Example: "book ticket".

\ P(w_i|w_{i-1})= \frac{Count(w_{i-1},w_i)} {Count(w_{i-1})} \

Trigram Model

It considers three consecutive words.

Example: "book two tickets".

\ P(w_i|w_{i-2},w_{i-1})= \frac{Count(w_{i-2},w_{i-1},w_i)} {Count(w_{i-2},w_{i-1})} \

4. Difference Between N-gram Models

Model	Context	Advantage
Unigram	One word	Simple
Bigram	Previous one word	Understands nearby word relationships
Trigram	Previous two words	Understands more context

5. Probability Calculation Example

Consider the following training sentences:

book two tickets

book one ticket

book two bus tickets

Total word tokens = 10.

Word counts:

Word	Count
book	3
two	2
tickets	2
one	1
ticket	1
bus	1

Unigram probability of "book":

\ P(book)=(3)/(10)=0.3 \

Bigram probability of "two" after "book":

\ P(two|book)=(2)/(3)=0.667 \

Trigram probability of "tickets" after "book two":

\ P(tickets|book,two)=(2)/(2)=1 \

6. Zero Probability Problem

Consider the sentence:

"Book three tickets."

If the bigram "book three" does not occur in the training corpus:

\ P(three|book)=0 \

Therefore, the probability of the complete sentence becomes zero.

This is called the zero-probability problem.

7. Sparsity Problem

Sparsity means many possible word combinations are not found in the training dataset.

Unigram models have less sparsity.

Bigram models have more sparsity.

Trigram models have even more sparsity.

8. Sentence Generation

Example outputs:

Model	Sample generated sentence
Unigram	tickets bus book
Bigram	book two tickets
Trigram	book two bus tickets

These are illustrative examples, not generated experimental samples.

Result: Unigram, bigram and trigram models estimate sentence probabilities using word counts. However, unsmoothed models suffer from zero probabilities and data sparsity.

TASK 3 – Smoothing and Discounting (5 Marks)

1. Introduction

Smoothing is a technique used in NLP to solve the zero-probability problem. It assigns a small probability to words or word combinations that are not present in the training data.

2. Why Smoothing is Required

In an unsmoothed model, an unseen word combination receives zero probability.

For example:

"Book three tickets."

If "book three" is not present in training data, its probability becomes zero.

Smoothing solves this problem by distributing some probability mass to unseen combinations.

3. Types of Smoothing

A. Add-One (Laplace) Smoothing

Add-one smoothing adds 1 to every word count.

Formula:

\ P(w_i|w_{i-1})= \frac{C(w_{i-1},w_i)+1} {C(w_{i-1})+V} \

Where:

C = Word combination count.

V = Vocabulary size.

Example:

Suppose:

Count(book, two) = 2

Count(book) = 3

Vocabulary size = 6

\ P(two|book)=(2+1)/(3+6) \

\ =(3)/(9)=0.333 \

B. Add-k Smoothing

Add-k smoothing adds a small value k instead of 1.

Formula:

\ P(w_i|w_{i-1})= \frac{C(w_{i-1},w_i)+k} {C(w_{i-1})+kV} \

Example: Let k = 0.5.

\ P(two|book)=(2+0.5)/(3+(0.5×6)) \

\ =(2.5)/(6)=0.417 \

4. Comparison of Smoothing Methods

Method	Description	Advantage
Add-One	Adds 1 to all counts	Simple

Add-k	Adds a small value k	More flexible
Good-Turing	Adjusts counts based on frequency	Handles unseen combinations

5. Good-Turing Discounting

Good-Turing smoothing estimates the probability of unseen words by reducing the counts of observed words.

Formula:

\ C^*=(C+1)\frac{N_{C+1}}{N_C} \

Where:

\C\ = Original count.

\N_C\ = Number of events occurring C times.

\C^*\ = Adjusted count.

Example frequency table:

Frequency (r)	**Number of events (\\(N_r\\))**
1	10
2	5
3	2
4	1

For an event occurring once:

\ C^*=2×(5)/(10)=1 \

This adjusted count is used to estimate probabilities.

6. Comparison of Smoothing Methods

Method	**Advantages**	**Limitations**
Add-One	Easy to implement	Discounts frequent words too much
Add-k	Adjustable	Requires choosing k
Good-Turing	Estimates unseen events	Needs reliable frequency statistics

7. Python Program

```
from collections import Counterwords = ["book", "bus", "ticket", "book", "ticket", "bus", "book"]counts = Counter(words)V = len(counts)N = len(words)print("Word Probabilities")for word in counts: laplace = (counts[word] + 1) / (N + V) print(word, round(laplace, 3))
```

Result: Smoothing methods reduce the zero-probability problem and improve language model predictions. Good-Turing smoothing is useful for estimating unseen word combinations.

TASK 4 – Backoff and Deleted Interpolation (4 Marks)

1. Introduction

Backoff and interpolation are techniques used to improve N-gram language models. They help the model predict words when longer word combinations are not available in training data.

2. Backoff Method

Backoff uses a lower-order model when a higher-order model cannot find the required word combination.

Example:

First, check the trigram model.

If the trigram is unavailable, check the bigram model.

If the bigram is unavailable, use the unigram model.

Example: To predict "tickets" after "book two":

If the trigram is unavailable, the model uses the bigram or unigram probability.

3. Interpolation Method

Interpolation combines unigram, bigram and trigram probabilities to predict the next word.

Formula:

\ P(w_i)=\lambda_1P(w_i)+ \lambda_2P(w_i|w_{i-1}) \

\ +\lambda_3P(w_i|w_{i-2},w_{i-1}) \

Where:

\ \lambda_1+\lambda_2+\lambda_3=1 \

Example:

Let:

\\lambda_1=0.2\

\\lambda_2=0.3\

\\lambda_3=0.5\

Suppose:

Unigram probability = 0.2

Bigram probability = 0.4

Trigram probability = 0.6

\ P=0.2(0.2)+0.3(0.4)+0.5(0.6) \

\ =0.04+0.12+0.30 \

\ P=0.46 \

4. Difference Between Backoff and Interpolation

Backoff	**Interpolation**
Uses lower-order models when needed	Combines all model orders
Checks higher-order model first	Uses all probabilities
Uses a backoff weight	Uses lambda weights
Suitable for unseen combinations	Uses information from multiple contexts

5. Katz Backoff

Katz backoff uses discounted probabilities and a normalising weight called alpha.

Alpha distributes the remaining probability mass among unseen word combinations.

Without proper normalisation, the total probability may not equal 1.

6. Stupid Backoff

Stupid backoff is a simple method that uses lower-order scores when higher-order combinations are unavailable.

It is easy to implement but does not produce properly normalised probabilities.

Therefore, its output is a score rather than a true probability.

7. Deleted Interpolation

Deleted interpolation estimates lambda weights using held-out training data.

It checks which N-gram model predicts the words correctly and assigns weights based on its performance.

Example illustrative weights:

Model	Lambda weight
Unigram	0.20
Bigram	0.30
Trigram	0.50
Total	1.00

Low-frequency contexts should generally depend more on lower-order models because their higher-order estimates are less reliable.

8. Comparison

Method	Main advantage
Katz Backoff	Handles unseen higher-order combinations
Stupid Backoff	Simple and fast
Linear Interpolation	Combines all N-gram orders
Deleted Interpolation	Estimates weights using held-out data

Result: Backoff and interpolation improve N-gram prediction by using shorter contexts and combining probabilities. Deleted interpolation helps select suitable weights for better prediction.

TASK 5 – Entropy, Cross-Entropy and Perplexity (4 Marks)

1. Introduction

Entropy, cross-entropy and perplexity are measures used to evaluate the performance of a language model. They help determine how well a model predicts the next word in a sentence.

2. Entropy

Entropy measures the uncertainty in a probability distribution.

Formula:

\ H(X)=-\sum P(x)\log_2P(x) \

Example:

Consider two distributions.

Word	Distribution A	Distribution B
Bus	0.5	0.9
Train	0.5	0.1

For Distribution A:

$$H=-[0.5\log_2(0.5)+0.5\log_2(0.5)]$$

$$H=1 \text{ bit}$$

For Distribution B:

$$H=-[0.9\log_2(0.9)+0.1\log_2(0.1)]$$

$$H=0.469 \text{ bits}$$

Observation: Distribution A has higher uncertainty, while Distribution B is more predictable.

3. Cross-Entropy

Cross-entropy measures how well the predicted probabilities match the actual words.

Formula:

$$H(P,Q)=-\sum P(x)\log_2Q(x)$$

A lower cross-entropy value indicates better prediction.

4. Perplexity

Perplexity measures how confused a language model is when predicting the next word.

Formula:

$$PP=2^{H}$$

Where H is cross-entropy measured in bits per word.

Example:

If cross-entropy is approximately 6.49 bits:

$$PP=2^{6.49}\approx90$$

A perplexity of 90 means the model has uncertainty equivalent to choosing among approximately 90 equally likely alternatives at each prediction step.

5. Comparison

Measure	Meaning

Entropy	Uncertainty in a distribution
Cross-Entropy	Difference between actual and predicted distributions
Perplexity	Difficulty of predicting the next word

6. Comparison of Models

The following is an illustrative results table. Replace these values with measurements from your held-out test dataset.

Model	**Cross-Entropy**	**Perplexity**
Unigram	7.00	128.00
Bigram	5.80	55.72
Trigram	5.20	36.76
Smoothed Trigram	4.80	27.86

7. Word Error Rate (WER)

WER measures the number of errors in a recognised sentence.

Formula:

\ WER=(S+D+I)/(N)×100 \

Where:

S = Substitutions.

D = Deletions.

I = Insertions.

N = Number of words in the reference sentence.

Example:

Reference: Book two tickets.

Recognised: Book three tickets.

There is one substitution.

\ WER=(1)/(3)×100=33.33\% \

Result: Entropy, cross-entropy and perplexity help evaluate language models. Lower perplexity generally indicates better prediction when the models use the same vocabulary and evaluation conditions.

TASK 6 – English Word Classes and Tagset Design (4 Marks)

1. Introduction

Part-of-Speech (POS) tagging is the process of assigning grammatical labels to words based on their roles in a sentence.

Example:

"Book two tickets."

Book – Verb

Two – Number

Tickets – Noun

2. Open and Closed Word Classes

Open Word Classes

These classes can accept new words.

Nouns

Verbs

Adjectives

Adverbs

Examples: bus, ticket, travel, quickly.

Closed Word Classes

These classes contain a limited number of words.

Pronouns

Prepositions

Conjunctions

Determiners

Auxiliary verbs

Examples: I, to, and, the, is.

3. Importance of Word Classes

Word classes help the system understand the grammatical role of each word.

For example:

"Book a ticket." – Book is a verb.

"Read the book." – Book is a noun.

Therefore, POS tagging is important for understanding transport enquiries.

4. Proposed Tagset

The following 25 tags are designed for the public transport enquiry domain.

Tag	Meaning	Penn Treebank Mapping
NN	Singular noun	NN
NNS	Plural noun	NNS
NNP	Proper noun	NNP
NNPS	Plural proper noun	NNPS
VB	Base verb	VB
VBD	Past verb	VBD
VBG	Gerund verb	VBG
VBN	Past participle	VBN
VBP	Present verb	VBP
VBZ	Third-person verb	VBZ
JJ	Adjective	JJ
JJR	Comparative adjective	JJR
JJS	Superlative adjective	JJS
RB	Adverb	RB
RBR	Comparative adverb	RBR
RBS	Superlative adverb	RBS
PRP	Personal pronoun	PRP
DT	Determiner	DT
IN	Preposition	IN
CC	Conjunction	CC
CD	Cardinal number	CD
MD	Modal verb	MD
UH	Interjection	UH
RP	Particle	RP
TO	To marker	TO

5. Ambiguous Words

Some words can have multiple grammatical roles.

Word	Sentence	POS Tag
Book	Book a ticket	VB
Book	Read the book	NN
Return	Return tomorrow	VB
Return	Return ticket	NN
Stand	Please stand here	VB
Stand	Bus stand	NN

6. Tag Frequency Distribution

Tag frequency is the number of times each tag appears in the annotated dataset.

Example:

Tag	Example frequency
NN	600
NNS	350
VB	400
NNP	300
JJ	250
RB	200
Other tags	900
Total	3000

These frequencies are illustrative and must be replaced with actual annotation counts.

7. Inter-Annotator Agreement

Inter-annotator agreement measures how consistently two people assign tags to the same words.

Percentage agreement:

\ Agreement=\frac{\text{Matching tags}}{\text{Total tags}}×100 \

Example:

If two annotators agree on 450 out of 500 tokens:

\ Agreement=(450)/(500)×100=90\% \

Cohen's Kappa:

\ \kappa=(P_o-P_e)/(1-P_e) \

Where:

\P_o\ = Observed agreement.

\P_e\ = Expected agreement by chance.

If \P_o=0.90\ and \P_e=0.50\:

\ \kappa=(0.90-0.50)/(1-0.50)=0.80 \

8. Residual Ambiguity

Residual ambiguity means a word can still have more than one possible tag.

Example:

Book – Noun or Verb.

Return – Noun or Verb.

Stand – Noun or Verb.

The ambiguity rate can be calculated using:

\ Ambiguity\ Rate= \frac{\text{Ambiguous word types}} {\text{Total word types}}×100 \

Result: A domain-specific tagset helps identify nouns, verbs, locations, numbers and other grammatical categories in transport enquiry sentences.

TASK 7 – Rule-Based Part-of-Speech Tagging (4 Marks)

1. Introduction

Rule-based POS tagging assigns grammatical labels to words using predefined rules.

It uses a dictionary and grammatical conditions to identify the correct tag.

2. Two-Stage Architecture

Stage 1: Dictionary

The dictionary stores words and their possible grammatical tags.

Example:

Word	**Possible tags**
Book	NN, VB
Return	NN, VB
Stand	NN, VB
Tickets	NNS

Stage 2: Rule Application

Rules are applied to remove incorrect tags and select the correct grammatical role.

3. Fifteen Disambiguation Rules

No.	Condition	Action
1	Word ends in -ing	Assign VBG
2	Word ends in -ed	Assign VBD or VBN using context
3	Word ends in -ly	Assign RB
4	Word ends in -ous	Assign JJ
5	Word ends in -tion	Assign NN
6	Word follows "the"	Prefer NN
7	Word follows "to"	Prefer VB
8	Word follows a modal verb	Assign VB
9	Word follows "is" and ends in -ing	Assign VBG
10	Word follows a determiner	Prefer NN
11	Word is a number	Assign CD
12	Word is a known station name	Assign NNP
13	Word follows a personal pronoun	Prefer VB
14	Word ends in -s and follows a determiner	Prefer NNS
15	Word is "and"	Assign CC

These are simple rules; contextual exceptions must be handled during implementation.

4. Python Program

def tag_word(word, previous): if word in ["book", "return", "stand"]: if previous in ["the", "a"]: return "NN" else: return "VB" elif word.endswith("ing"): return "VBG" elif word.endswith("ly"): return "RB" elif word.isdigit(): return

"CD" else: return "NN"sentence = "book two tickets"words = sentence.split()for i, word in enumerate(words): previous = words[i-1] if i > 0 else "" print(word, tag_word(word, previous))

5. Evaluation

Accuracy formula:

\ Accuracy=\frac{\text{Correct predictions}} {\text{Total predictions}}×100 \

Example:

If 850 out of 1000 words are tagged correctly:

\ Accuracy=(850)/(1000)×100=85\% \

Accuracy on ambiguous words should also be measured separately because those words are more difficult to classify.

6. Common Errors

Error	**Example**
Noun predicted as verb	Book
Verb predicted as noun	Return
Unknown station name	Vellore
Incorrect suffix rule	Interesting
Context error	Stand

7. Rule Conflict

Example:

"Book the ticket."

Rule 7 may assign VB to a word after "to".

Rule 10 assigns NN to a word after a determiner.

When two rules conflict, the system follows the rule priority and context. Proper rule ordering is important for accurate tagging.

Result: Rule-based tagging uses dictionaries and grammatical rules to identify parts of speech. It is easy to understand and maintain for simple cases but may fail when words have multiple meanings.

TASK 8 – Stochastic (HMM) Tagging and the Viterbi Algorithm (5 Marks)

1. Introduction

Stochastic POS tagging uses probability to identify the correct grammatical tag of a word.
A Hidden Markov Model (HMM) is a statistical model that predicts the most likely sequence of POS tags for a sentence.
Example:
"Book two tickets."
Book – VB
Two – CD
Tickets – NNS

2. HMM Tagging Equation

Using Bayes' rule:
\ P(T|W)=(P(W|T)P(T))/(P(W)) \
Where:
T = Sequence of POS tags.
W = Sequence of words.
P(T) = Probability of tag sequence.
P(W|T) = Probability of words given tags.
The HMM chooses the tag sequence with the highest probability.

3. Two Independence Assumptions

Transition assumption: The current tag depends only on the previous tag.
Emission assumption: The current word depends only on its current tag.
These assumptions make the model simpler and faster.

4. Transition and Emission Probabilities

Transition Probability
It measures the probability of one tag following another.
\ P(t_i|t_{i-1})= \frac{Count(t_{i-1},t_i)} {Count(t_{i-1})} \
Emission Probability
It measures the probability of a word occurring with a particular tag.
\ P(w_i|t_i)= \frac{Count(t_i,w_i)} {Count(t_i)} \

Example:

Previous Tag	Current Tag	Probability
DT	NN	0.80
VB	CD	0.60
CD	NNS	0.90

These are illustrative probabilities.

5. Viterbi Algorithm

Viterbi is a dynamic programming algorithm used to find the most likely sequence of hidden states.

Steps:

Initialise the probability of each possible first tag.

Calculate transition and emission probabilities.

Find the maximum probability for each tag.

Store the previous tag using a back-pointer.

Repeat for every word.

Select the highest final probability.

Trace back to obtain the best tag sequence.

6. Viterbi Example

Sentence:

"Book two tickets now."

Possible tags:

Book – VB, NN

Two – CD

Tickets – NNS

Now – RB

Best sequence:

Word	Selected tag
Book	VB
Two	CD
Tickets	NNS

Now	RB

The final tag sequence is:

\ VB → CD → NNS → RB \

A complete Viterbi calculation requires transition and emission values from the annotated training corpus. The following is a simplified illustration of the trellis.

Word	**Candidate tag**	**Illustrative probability**
Book	VB	0.40
Book	NN	0.20
Two	CD	0.60
Tickets	NNS	0.80
Now	RB	0.70

7. Comparison of Tagging Methods

Method	**Description**
Most-Frequent-Tag	Assigns the most common tag
Exhaustive Search	Checks every possible tag sequence
Viterbi	Efficiently finds the best sequence using dynamic programming

For a sentence with n words and k possible tags, exhaustive search may check up to \k^n\ sequences, while Viterbi takes approximately \O(nk^2)\ operations.

8. Unknown Word Handling

Unknown words are words that are not found in the training vocabulary.

Example: A newly introduced station name.

The HMM can handle unknown words using:

Suffix information.

Capitalisation.

Word shape.

Open-class tag probabilities.

Example:

A capitalised unknown word such as "Vellore" may be assigned the proper noun tag NNP, especially when it occurs after "to".

Result: HMM tagging uses transition and emission probabilities to identify the most likely POS sequence. The Viterbi algorithm makes the prediction efficient.

TASK 9 – Transformation-Based (Brill) Tagging (4 Marks)

1. Introduction

Transformation-based tagging is a POS tagging method that improves initial tagging by applying correction rules.

It is also called Brill tagging.

It learns rules from training data and uses them to correct tagging errors.

2. Working Procedure

Assign an initial tag to every word.

Compare predicted tags with correct tags.

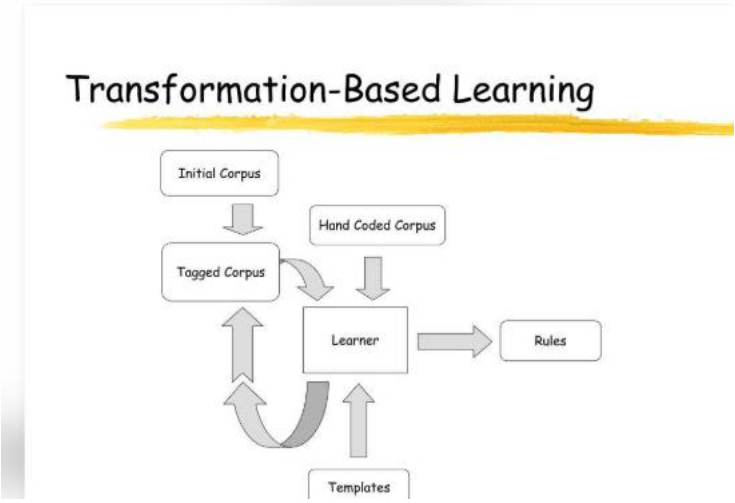
Identify tagging errors.

Generate possible correction rules.

Select the rule that corrects the maximum errors.

Apply the selected rule.

Repeat until no useful rule remains.



3. Rule Templates

The following six templates can be used:

No.	Rule template
1	Change tag if previous tag is NN
2	Change tag if previous tag is VB
3	Change tag if next tag is NN
4	Change tag if next word is "ticket"
5	Change tag if previous word is "to"
6	Change tag if the word ends with "ing"

These rules use the surrounding words and grammatical context.

4. Example of Transformation

Sentence:

"Book the ticket."

Initial tagging:

Word	Initial tag
Book	NN
the	DT
ticket	NN

After applying the correction rule:

Word	Final tag
Book	VB
the	DT
ticket	NN

The system changes Book from NN to VB because it is used as an action.

5. First Ten Transformations

The following table illustrates possible learned rules. Actual rules and error counts must come from running the Brill learner.

Rule	Transformation	Errors corrected	Errors introduced
1	NN → VB after "to"	20	2
2	NN → VBG for -ing	18	3

	words		
3	VB → NN after "the"	15	2
4	NN → NNP for known stations	14	1
5	NN → RB for -ly words	12	2
6	NN → NNS after determiners	10	2
7	NN → VB after modal verbs	9	1
8	VB → VBD for -ed words	8	2
9	NN → CD for numbers	7	0
10	NN → JJ before nouns	6	1

6. Advantages

Learns correction rules automatically.

Easy to understand.

Improves initial tagging.

Rules can be examined by humans.

Uses grammatical context.

7. Limitations

Requires correctly annotated training data.

May learn incorrect rules from limited data.

Too many rules can cause overfitting.

Training can take more time.

8. Accuracy Curve

Accuracy can be measured after applying different numbers of transformation rules.

Example:

Number of rules	Illustrative accuracy
0	75%
2	80%
4	84%
6	87%
8	88%
10	87%

The best point is where the development accuracy is highest. If accuracy decreases after additional rules, overfitting may have occurred.

Result: Brill tagging improves POS tagging by learning correction rules. It is readable and understandable but requires careful rule selection to avoid overfitting.

TASK 10 – Implement and Test the Models (3 Marks)

1. Introduction

Implementation means converting the theoretical NLP concepts into working Python programs.

Python and NLTK can be used to build and test N-gram models and POS taggers.

2. Software Requirements

Software	Purpose
Python	Programming
NLTK	NLP processing
Matplotlib	Graph generation
Collections	Word counting
Google Colab / Python IDLE	Program execution

3. Python Program – Word Counting

from collections import Countersentences = ["book two tickets", "book one ticket", "check bus timing", "book two bus tickets"]words = []for sentence in sentences:

words.extend(sentence.lower().split())counts = Counter(words)print("Word Frequencies:")for word, count in counts.items(): print(word, count)print("Vocabulary Size:", len(counts))print("Total Tokens:", len(words))

4. Python Program – Bigram Model

from collections import Countersentence = "book two tickets to vellore"words = sentence.split()bigrams = []for i in range(len(words)-1): bigrams.append((words[i], words[i+1]))print("Bigram Model:")for bigram, count in Counter(bigrams).items(): print(bigram, count)

5. Python Program – POS Tagging

import nltksentence = "Book two tickets"words = nltk.word_tokenize(sentence)tags = nltk.pos_tag(words)print("POS Tags:")for word, tag in tags: print(word, tag)

If NLTK data is not installed, run:

import nltknltk.download('punkt')nltk.download('punkt_tab')nltk.download('averaged_perceptron_tagger_eng')

6. Testing

Testing checks whether the program produces the expected output.

Test Case	Input	Expected Output
1	book two tickets	Three tokens
2	book two	One bigram
3	Book	NN or VB based on context
4	123	CD
5	booking	VBG

7. Performance Evaluation

The following is an illustrative comparison. Actual time, memory and accuracy must be measured during execution.

Model	Performance measure
Unigram	Simple and fast

Bigram	Uses previous word
Trigram	Uses more context
Rule-Based	Depends on rules
HMM	Uses probability
Brill	Uses correction rules

8. Graphs

Two important graphs should be included:

Perplexity versus N-gram order.

Tagging accuracy versus training dataset size.

Result: Python implementation helps build, test and compare NLP models. Testing ensures that the programs produce the expected results.

TASK 11 – Compare, Recommend and Defend the Design (2 Marks)

1. Introduction

Different language models and POS taggers have different advantages and limitations. Comparing them helps select the best model for the public transport enquiry helpline.

2. Comparison of Language Models

Model	Advantage	Limitation
Unigram	Simple and fast	Does not understand context
Bigram	Uses previous word	Cannot understand long context
Trigram	Uses two previous words	Data sparsity
Smoothed Trigram	Handles unseen combinations	Requires smoothing

3. Comparison of POS Taggers

Tagger	Advantage	Limitation
Rule-Based	Easy to understand	Requires manual rules
HMM	Uses probability and context	Unknown words are difficult

Brill	Learns correction rules	Can overfit

4. Recommended Models

Recommended Language Model: Smoothed Trigram with Backoff

Reasons:

Uses more word context.

Handles unseen word combinations.

Improves sentence prediction.

Helps select better speech recognition outputs.

Recommended POS Tagger: HMM with Viterbi Algorithm

Reasons:

Uses transition and emission probabilities.

Predicts complete tag sequences.

Handles ambiguous words using context.

Efficient for processing large numbers of sentences.

These are design recommendations; final selection should be confirmed using measured perplexity, WER, accuracy, speed and memory.

5. Limitations

Unknown station names may cause errors.

Code-mixed sentences are difficult to understand.

Incorrect speech recognition can affect tagging.

More training data is required for better accuracy.

6. Conclusion

The smoothed trigram model with backoff and HMM-based POS tagging are suitable starting choices for the transport enquiry helpline. Their performance should be verified using actual test data before deployment.

TASK 12 – Day-4 Design Challenge: Re-evaluate and Adapt the Design (8 Marks)

Important – 8 Marks

A. Impact Analysis (2 Marks)

1. Introduction

The transport enquiry helpline is extended to metro and intercity railway services. New station names, train names and fare categories are introduced.

Callers also start using code-mixed languages such as Telugu and English.

Example:

"Chennai to Madurai bus timing enna."

More than 15% of the words in new calls are not present in the original training vocabulary.

2. Problems in the Original Design

Vocabulary Problem

New station names are not available in the old vocabulary.

Example: New railway station names may be treated as unknown words.

OOV Problem

The number of unknown words increases because new words are continuously introduced.

Code-Mixing Problem

The original model mainly understands English words. It may not understand Telugu-English mixed sentences.

Tagset Problem

The original tagset does not contain separate categories for code-mixed words.

Probability Problem

Old transition and emission probabilities may not work correctly for new railway enquiries.

3. Impact Analysis Table

Changed Requirement	Impact
New station names	Increased OOV rate
Metro services	New vocabulary
Intercity trains	New train names
Code-mixed speech	Tagging errors
New fare categories	Prediction errors
More than 15% unknown words	Increased perplexity

4. Why OOV Words Are Important

Station names are very important in transport enquiries because they identify the origin and destination.

Example:

"Bus from Chennai to Vellore."

If Vellore is treated as an unknown word, the system may fail to identify the correct destination.

Therefore, unknown station names must be handled carefully.

5. Effect on Model Performance

Perplexity increases because more words are unknown.

Tagging accuracy decreases because new words cannot be identified correctly.

Speech recognition rescoring becomes less reliable.

Destination extraction may fail.

Result: The changed requirements reduce the performance of the original language model and POS taggers.

B. Evaluation of the Original Design (2 Marks)

1. Introduction

The original language model and taggers must be tested using new metro and railway enquiry sentences.

2. Example Test Sentences

Sentence 1 – Unseen Station Name

"Book a ticket to Kanchipuram."

Problem: Kanchipuram may not be present in the original vocabulary.

Sentence 2 – Code-Mixed Sentence

"Chennai to Madurai bus timing enna."

Problem: The model may not recognise the Telugu-English or other mixed-language structure. Here, enna is a non-English word.

Sentence 3 – Both Problems

"Book train ticket to new station Vijayawada."

Problem: The model may not recognise new station names and railway-related vocabulary.

3. Evaluation Table

Test Sentence	Problem	Expected Failure
Book a ticket to Kanchipuram	Unseen station	OOV problem
Chennai to Madurai bus timing enna	Code-mixing	Language identification problem
Book train ticket to Vijayawada	New domain vocabulary	Vocabulary and tagging problem

4. Performance Comparison

The following table shows the type of degradation expected. Actual values must be obtained by evaluating both datasets.

Evaluation	Original system	Extended system
Perplexity	Lower on old domain	May increase
OOV rate	Lower	Above 15%
Tagging accuracy	Higher on old domain	May decrease
WER	Lower on old domain	May increase

5. Which Component Degrades Fastest?

The language model may degrade quickly because many new station and train names are missing from its vocabulary.

The POS tagger may also fail because it cannot correctly identify new proper nouns and code-mixed words.

6. Better Approaches

Class-based N-gram model.

Character-level language model.

Station-name gazetteer.

Open-vocabulary POS tagger.

Code-mixed language processing.

Result: The original model has limitations when processing unseen station names and code-mixed sentences. Additional vocabulary and language support are required.

C. Adaptation or Redesign (2 Marks)

1. Proposed Solution

I recommend MODIFYING the original design instead of completely replacing it.
The existing N-gram and HMM models can still be useful, but they need additional features to support new railway and metro enquiries.

2. Proposed Changes

Update Vocabulary
Add new bus stops, metro stations, railway stations, train names and fare categories.
Add Station Gazetteer
A gazetteer is a collection of known geographical names.
Example:
Chennai
Madurai
Vijayawada
Hyderabad
Kanchipuram
It helps identify unknown station names.
Add Character N-gram Model
Character N-grams help identify words based on their character patterns.
Example:
Vijayawada
Even if the complete word is not available, character patterns can provide useful information.
Extend the Tagset
Add new categories for code-mixed words and domain-specific entities.
Example:

New Tag	Meaning
LOC	Location or station name
TRAIN	Train name

LANG	Code-mixed language token
FARE	Fare category

These are additional domain labels and need to be mapped to the existing tagset consistently.

Update Smoothing

Use Good-Turing or suitable backoff smoothing to handle unseen combinations.

Retrain the Model

Use new metro and railway enquiry data to update the model probabilities.

3. Modified System Architecture

Speech Recognition Input

Bus, metro and railway enquiries

Text Preprocessing

Tokenisation and language identification

Updated Vocabulary and Gazetteer

Station names, train names and fare details

Character N-gram

Unknown word handling

Smoothed N-gram

Sentence prediction

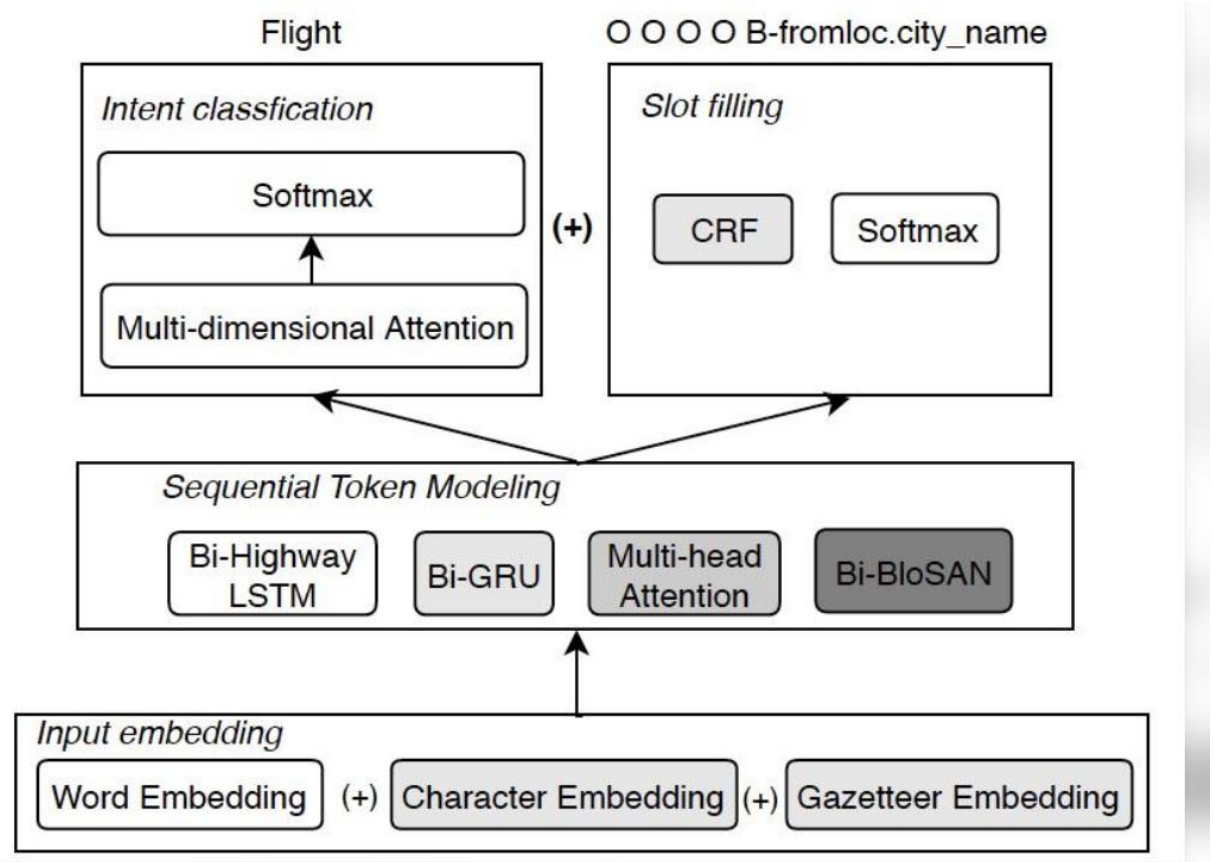
Modified HMM POS Tagger

Viterbi algorithm and extended tagset

Enquiry Information Extraction

Origin, destination, date, time and fare

Final Response to Caller



4. Expected Effect of Adaptation

Parameter	**Expected improvement**
Vocabulary	Covers more new words
OOV handling	Better recognition of unknown names
Perplexity	Expected to decrease
Tagging accuracy	Expected to improve
Retraining	Requires regular updates
Annotation effort	Increases initially

These are expected effects, not measured results.

5. Retraining Frequency

Update the station gazetteer whenever new stations are introduced.

Collect new enquiry sentences continuously.

Retrain or update model probabilities monthly.

Evaluate the updated model before deployment.

Result: Modifying the original design with a gazetteer, character N-grams, improved smoothing and an extended tagset helps handle new transport vocabulary and code-mixed enquiries.

D. Evidence-Based Defence (2 Marks)

1. Final Decision

I recommend modifying the original system because the existing N-gram and HMM approaches are useful, but the new requirements introduce unknown words and code-mixed language.

2. Reasons for the Decision

A station-name gazetteer helps recognise new geographical names.

Character N-grams help process unseen words.

Smoothing reduces zero-probability problems.

An extended tagset supports new word categories.

Retraining improves performance using updated data.

3. Limitations of the Modified System

Some unknown words may still be incorrectly identified.

Code-mixed language may have complex grammatical structures.

Incorrect speech recognition can affect the final output.

N-gram models mainly use limited word context.

They cannot fully understand long-distance relationships or the complete meaning of a conversation.

4. Final Recommendation Table

Component	**Recommended Method**
Language Model	Smoothed Trigram with Backoff
Unknown Word Handling	Character N-gram
Station Recognition	Gazetteer

POS Tagging	HMM with Viterbi
Tagset	Extended Domain-Specific Tagset
Model Update	Monthly retraining and continuous gazetteer updates

5. Conclusion

The original NLP system is suitable for basic bus and train enquiries, but it requires modifications to support metro services, new station names and code-mixed language.

The modified system provides better vocabulary coverage and more reliable word tagging. Therefore, modifying the existing design is a practical and economical solution for the extended public transport enquiry helpline.